PRACTICE1 / ТЕХНИЧЕСКИЙ ОТЧЁТ / 01

Резервное копирование Kubernetes-приложения в S3

Локальный стенд, восстановление в чистом namespace и проверка целостности

Проект practice1 • 02 ноября 2026 года • Windows / Docker / kind

ПОЛНЫЙ ПРОГОН: PASS

Цель работы - создать воспроизводимый цикл защиты приложения: подготовить контрольные данные PostgreSQL, сохранить объекты Kubernetes и согласованный дамп в S3, удалить исходное приложение, восстановить его в новом пространстве имён и проверить результат на уровне данных и API.

Основной результат - исходный код стенда, манифесты, сценарии штатных и аварийных операций, эксплуатационная инструкция и реальные журналы с длительностями. Облачный вариант VK Cloud описан отдельно; учётные данные облака не использовались. Публикацию локального Git-репозитория выполняет пользователь.

Состав отчёта

Страница	Раздел
2	Архитектура и границы стенда
3	Данные и согласованность копии
4	Развёртывание и конфигурация
5	Ручное и плановое копирование
6	Восстановление и критерии приёмки
7	Проверка отказов
8	Секреты и подключение VK Cloud
9	Фактические измерения
10	Инструкция сдачи, ограничения и источники

Архитектура стенда

На компьютере запущен один узел kind. Отдельный kubeconfig и явный контекст kind-practice1 изолируют сценарии от других кластеров пользователя. Локальный provisioner создаёт PV типа local; размер в PVC задаёт запрос ресурса, но для локального хранилища не заменяет контроль свободного места на диске.

Область	Компоненты и назначение
practice1-app	PostgreSQL, API PostgREST, Service, Secret, ConfigMap; PVC pgdata (2 GiB) и backup-data (1 GiB).
practice1-storage	MinIO, Service ClusterIP, PVC 5 GiB, Secret администратора и технических пользователей, служебные Job mc.
velero	Сервер Velero, node-agent, AWS plugin, BackupStorageLocation, Schedule, Backup, Restore, PVB/PVR.
practice1-restore-*	Новый namespace с восстановленными объектами, новым PGDATA и импортированным дампом.
Хост: practice1	Манифесты, Python-сценарии, отчёт и журналы; приватное состояние в work/.

Поток резервирования

PostgreSQL → pg_dump → PVC backup-data → Velero FSB / Kopia → S3 MinIO.
Объекты Kubernetes → Velero AWS plugin → тот же приватный бакет.
S3 → новый PVC backup-data → проверка SHA-256 → pg_restore → новый PostgreSQL → SQL + HTTP-проверка.

Снимки облачных дисков отключены. Аннотация Pod явно выбирает только backup-data для файлового резервирования. Описания PVC входят в копию, физические PV и их привязка к исходному узлу не переносятся. StorageClass заранее установлен на целевом кластере, а восстановление создаёт новые тома.

Restore использует ConfigMap resource modifiers: из PVC удаляются старый volumeName и аннотации привязки. Без этого при исключённых физических PV ссылки на удалённые тома оставляли PVC в Pending.

Граница доказанного восстановления

MinIO находится в том же кластере, но вне namespace приложения. Это позволяет проверить удаление приложения и его данных, сохранив резервную копию. Потеря всего kind-кластера, диска или компьютера уничтожит и локальный MinIO. Для такой аварии нужен внешний S3, отдельно сохранённые ключи и пароль репозитория. Отказоустойчивость production-системы здесь не заявляется.

Основание: документация Kubernetes PV, Velero FSB и Rancher Local Path Provisioner; см. источники 1, 6, 7.

Контрольные данные и согласованность

Параметр	Значение
accounts	100 строк: id и уникальное имя account-NNN
orders	1000 строк: id, внешний ключ account_id, amount_cents, description
Сумма amount_cents	8 508 500; целочисленные копейки исключают ошибки float
Ограничения	2 primary key, 1 unique, 1 foreign key, 1 check
Sequence	orders_id_seq: last_value=1000; is_called=true
Эталон SHA-256	f6d09d7a89142a87847c6cf431f71d37786657dd9d36b0b11c2157cdcea7a2b1

seed.sql выполняется только при первоначальной подготовке пустой базы. API использует роль reader с правом SELECT и default_transaction_read_only. Во время контрольного эксперимента писателей нет; это делает сравнение до и после копии однозначным. При появлении production-писателей контрольный эталон должен вычисляться из того же snapshot, что и дамп, либо должна вводиться пауза записи.

Почему выбран логический дамп

PostgreSQL pg_dump создаёт согласованный архив одной базы. Формат custom используется для pg_restore; параметры no-owner и no-acl отделяют данные от кластерных ролей. Роль reader создаётся отдельно при инициализации. Глобальные роли, tablespace и PITR/WAL не входят в этот вариант копии. Файловое копирование активного PGDATA здесь не применяется.

Два разных контроля целостности

Первый SHA-256 относится к файлу app.dump: он проверяется перед импортом и обнаруживает повреждение архива. Второй SHA-256 относится к данным: таблицы выгружаются в CSV с фиксированным набором столбцов и ORDER BY id; строки нормализуются по LF, таблицы разделяются метками. Такой хеш обнаруживает изменение значения даже при сохранении общего числа строк и суммы.

Дополнительно сравниваются количество accounts/orders, сумма, ограничения схемы, sequence и ответ /orders?order=id&limit=1. В unit-тесте специально меняются местами суммы двух строк: агрегат остаётся равным, а SHA-256 различается. Это проверяет смысл контрольной процедуры.

Основание: PostgreSQL 17 pg_dump/pg_restore; контрольный набор и алгоритм хеширования определены кодом проекта.

Развёртывание и конфигурация

Компонент	Закреплённая версия / режим
kind / Kubernetes	v0.33.0 / v1.35.8
Velero / AWS plugin	v1.18.3 / v1.14.2
PostgreSQL / API	postgres:17.6-bookworm / postgrest/postgrest:v12.2.3
MinIO / mc	Сборка официальных исходников из релизов 22.04.2025 / 16.04.2025
FSB	Kopia, node-agent, opt-in по аннотации, snapshots=false

На рабочей станции первоначально отсутствовали Docker, kind, kubectl и Velero. Docker Desktop установлен в профиль пользователя с WSL2 и запущен без перезагрузки. CLI загружены в tools/bin с проверкой SHA-256 издателя. Node image kind закреплён digest, а версия приложения - явными тегами.

Первый запуск обнаружил отказ реестра при загрузке Community-образов MinIO. Репозиторий MinIO описывает распространение исходным кодом. Поэтому проект содержит Dockerfile.minio и Dockerfile.mc: сборка из официального Git-тега, затем kind load docker-image. Повторный bootstrap использует локальный кеш. Исходники не модифицируются; лицензия сохраняется в образе.

Последовательность bootstrap

Проверка Linux Docker → кластер и namespaces → StorageClass → сборка образов → случайные ключи и Secret → MinIO и бакет → Velero и node-agent → проверка BSL Available → приложение → заполнение БД → проверка API и PV → ежедневное расписание.

```
python scripts/install_tools.py
python -m pip install -r requirements.txt
.\run.ps1 bootstrap
```

Velero запускается с provider=aws, s3ForcePathStyle=true, локальным s3Url и region=us-east-1. Создаётся отдельный технический пользователь MinIO, которому доступны операции только в бакете practice1-backups. У root-пользователя остаются административные операции создания бакета и политик.

Ошибки любого шага дают ненулевой код завершения, файл result.json: FAIL и диагностику Pod/Event. Частичный запуск не помечается успешным. Повторный запуск не перезаполняет уже существующую контрольную БД.

Ручная и плановая резервные копии

Backup включает только practice1-app. Из копии исключаются events, jobs и persistentvolumes; includeClusterResources=false, snapshotVolumes=false, defaultVolumesToFsBackup=false. PVC backup-data выбирается аннотацией Pod. Ручной Backup и Schedule используют один шаблон.

Pre-hook и post-hook

Pre-hook получает атомарную блокировку каталогом .dump-lock, запускает pg_dump во временный файл, проверяет читаемость оглавления и заменяет app.dump. Затем записываются app.dump.sha256, dump-bytes.txt и dump-created-utc.txt. Post-hook освобождает блокировку после чтения тома Velero. on-error=Fail запрещает считать ошибку подготовки успешной копией.

Механизм	Параметры / проверка
Ручной запуск	.\run.ps1 backup; уникальное имя Backup; ожидание терминальной фазы.
Ежедневный Schedule	0 23 * * * UTC; 02:00 по Москве следующего дня; TTL 168 часов.
Проверка планировщика	Отдельный Schedule * * * * *; фактически созданный контроллером Backup; затем пауза и очистка Schedule.
Готовность копии	Completed, errors=0, ровно один завершённый PVB backup-data и totalBytes > 0.
Фактический размер дампа	11478 байт
Бакет после ручной копии	219357 байт; 76 объектов (включая предыдущие копии)

После завершения сохраняются JSON Backup/PVB, журнал Velero, размер дампа и инвентаризация бакета mc ls с mc du. Размер дампа, объём прочитанных FSB-данных и полный размер бакета различны: S3 также содержит метаданные, журналы и служебные объекты Kopia. Дедупликация влияет на физически занятый объём.

Условия допустимого удаления

Скрипт повторно проверяет состояние копии и данные источника, а также метку принадлежности namespace стенду. После этого приостанавливает ежедневный Schedule и удаляет practice1-app. Готовность одной лишь копии Kubernetes-объектов недостаточна: отсутствие PVB запрещает удаление.

Срок хранения управляется Velero. Самостоятельное удаление блоков Kopia или несогласованный S3 lifecycle могут повредить несколько дедуплицированных копий. Для очистки используются штатные операции Velero.

Восстановление и критерии приёмки

Restore создаётся только для отсутствующего namespace с префиксом practice1-restore-. namespaceMapping переносит practice1-app в целевое имя. Исходный namespace к этому моменту удалён. Восстановление в существующую среду запрещено, чтобы не смешивать старые и восстановленные данные.

Порядок восстановления

1. Velero читает описания объектов и восстанавливает PVC, конфигурацию, Secret и приложение.
2. Provisioner выдаёт новые локальные PV. FSB восстанавливает /backup.
3. Init-контейнер Velero пропускает запуск приложения после завершения переноса тома.
4. PostgreSQL инициализирует пустой PGDATA; restore.sh проверяет файл и импортирует архив одной транзакцией.
5. После Ready выполняются SQL-проверки и HTTP-запрос к API.

Уровень	Условие приёмки
Velero	Restore Completed, errors=0; PodVolumeRestore существуют и все Completed.
Контейнеры	PostgreSQL Ready; Deployment API доступен; лог содержит RESTORE_IMPORT_COMPLETED.
Файл	sha256sum --check app.dump.sha256 завершается успешно до импорта.
Данные	Хеш совпадает с эталоном; 100 accounts, 1000 orders, сумма 8 508 500.
Схема	5 ограничений; sequence 1000|t.
Приложение	Первый заказ: id=1, account_id=1, amount_cents=17, description=order-0001.

Если PostgreSQL уже имеет данные, штатный entrypoint не выполняет импорт повторно. Наличие маркера в журнале и запрет существующего namespace не позволяют выдать такой запуск за проверку чистого восстановления. seed.sql после Restore не вызывается ни явно, ни через init-скрипты.

Фактическое сравнение: SHA-256 и контрольные данные совпали.

В завершённых Restore зарегистрированы 3 предупреждения: два об исключённых физических PV и одно об уже существующей kube-root-ca.crt. Они проверены: новые PVC Bound, том восстановлен, данные и API верны. Ненулевое число ошибок при этом недопустимо.

Для этого эксперимента RTO определяется от начала операции Restore до конца проверки данных/API, включая предварительную проверку копии. Это более строгая граница, чем время появления фазы Completed. Плановый RPO зависит от интервала расписания и возраста последней действительно успешной копии.

Проверка отказов и действия оператора

Сбой	Инъекция и ожидаемый сигнал	Восстановление работы
Нет бакета	Уникальное отсутствующее имя, правильные ключи; BSL Unavailable, NoSuchBucket.	Проверить endpoint/регион/имя; создать бакет; дождаться Available.
Неверный ключ	Несуществующий Access Key ID; InvalidAccessKeyId либо отказ доступа.	Обновить пару ключей через Secret; повторить проверку BSL.
Нет разрешений	Реальный пользователь с Deny-политикой; AccessDenied/Forbidden.	Исправить bucket/object права; учесть явный Deny.
Прерван Restore	Node-agent временно заблокирован; активный Restore и PVR; сервер Velero перезапущен.	Вернуть agent; сохранить логи; удалить частичный namespace; новый Restore.

Три теста S3 создают отдельные BackupStorageLocation и Secret. После проверки диагностического сообщения создаётся реальный Backup в неисправную BSL: ожидается отказ, а не Completed. Временные BSL/Secret очищаются, рабочая default должна остаться Available. Это проверяет реакцию Velero, а не только возможность выполнить команду клиента S3.

Как исключается ложная проверка прерывания

Перед перезапуском сервера сценарий обязан увидеть активный Restore и PodVolumeRestore. Блокировка node-agent оставляет окно для детерминированной инъекции. Если Restore успел завершиться раньше, тест считается неуспешным. finally возвращает исходный nodeSelector и реплики сервера. Затем выполняется повторное восстановление в новом чистом namespace с полной проверкой данных.

Перезапуск сервера Velero прерывает его текущие операции. Такой тест допустим в выделенном учебном кластере, где не выполняются сторонние копии. Состояние Failed/PartiallyFailed не означает, что восстановленные объекты автоматически удалены: очистка частичной среды выполняется отдельно.

При любой ошибке

Не переключать пользователей на неподтверждённую БД. Сохранить Restore/Backup/PVB/PVR JSON, логи Velero и PostgreSQL, события Pod, время начала и завершения. Устранить причину и использовать новый namespace. Наличие файлов в S3 не заменяет проверку восстановления.

Секреты и подключение VK Cloud

Секреты генерируются криптографически случайно; в репозиторий входят только ссылки secretKeyRef. В kubectl данные передаются через stdin, а payload и секретные ответы не записываются в журнал команд. Приватные файлы, kubeconfig и сырые журналы исключены из Git. Перед публикацией проверяются отслеживаемые файлы и содержимое архива.

Не публиковать	Почему / где хранится
Пароли PostgreSQL и API	Secret приложения и приватное work/; API URI также содержит пароль.
S3 access/secret key	Secret Velero; отдельная пара ключей для каждого хранилища.
MinIO root credentials	Только административные Job; рабочему Velero не выдаются.
Пароль Kopia repository	Нужен для чтения репозитория после потери установки; сохранить отдельно.
kubeconfig и дампы	Доступ к кластеру и содержимое базы; не отправлять в Git/публичный архив.

Secret Kubernetes - кодирование base64, а не шифрование. Secret приложения входит в резервную копию объектов. Шифрование данных Kopia не следует распространять как обещание на все файлы Velero в бакете. Нужны приватность S3, TLS, защита ключей, ACL и шифрование хранилища.

Рабочий стол расположен в OneDrive: исключение из Git не отключает синхронизацию ОС. Для production-ключей приватный каталог следует вынести из синхронизируемой папки. Локальный HTTP MinIO и привилегированный node-agent являются ограничениями учебной среды.

Вариант VK Cloud

Создать приватный бакет и технический S3-аккаунт. В config/vk-cloud-bsl.yaml указать свой бакет, endpoint и регион. Пример Москвы: https://hb.ru-msk.vkcloud-storage.ru, region=ru-msk; значения сверяются с текущим проектом. Secret vk-s3 создаётся из приватного AWS credentials-файла.

После применения отдельной BSL vk-cloud дождаться Available, создать малую копию с storageLocation=vk-cloud и выполнить тест Restore. Требуются ListBucket/GetBucketLocation и операции Get/Put/Delete/multipart над объектами своего бакета. TLS-проверку не отключать. Подключение к VK Cloud в данном прогоне не выполнялось; инструкция и шаблон предоставлены.

Фактические журналы и измерения

Источник измерений: demo-1102123502-8e91. Длительности получены monotonic-таймером Python; метки времени записаны в UTC. На рабочей станции использовалось московское время UTC+3.

Операция	Статус	Секунды
Проверка окружения	PASS	0.672
Создание / проверка kind	PASS	2.547
Сборка / загрузка MinIO	PASS	14.359
Создание Secret	PASS	1.156
MinIO и бакет	PASS	4.375
Установка Velero	PASS	5.500
Приложение и контрольные данные	PASS	29.657
Ежедневный Schedule	PASS	0.265
Проверка плановой копии	PASS	99.922
Ручная копия и размер S3	PASS	11.547
Сбой: нет бакета	PASS	1.734
Сбой: неверный ключ	PASS	1.641
Сбой: запрет доступа	PASS	1.640
Удаление исходного namespace	PASS	17.063
Восстановление в новый namespace	PASS	32.219
Проверка данных, схемы, API	PASS	1.422
Прерывание Restore и очистка	PASS	62.672
Восстановление в новый namespace	PASS	32.469
Проверка данных, схемы, API	PASS	1.593

RTO до подтверждённого приложения: 33.641 с, 34.062 с. Размер логического дампа: 11478 байт.

Таблица отражает один выбранный прогон, а не статистическую оценку производительности. Повторная сборка использует кеш, скачивание образов и первая инициализация увеличивают длительность. Исходные JSON/CSV и безопасные выдержки журнала включены в комплект сдачи.

Дополнительно выполнены 13 unit-проверок; все прошли. Проверен синтаксис Bash-скриптов. Unit-проверки дополняют реальное восстановление, а не заменяют его.

Эксплуатация, сдача и источники

Повторение и передача результата

Полный цикл: .\run.ps1 demo. Отдельные операции: bootstrap, backup, scheduled-proof, destroy-app, restore --target practice1-restore-check, verify, failures, interrupt. Завершённый demo оставляет восстановленное приложение и MinIO доступными в кластере, а ежедневное расписание исходного namespace - на паузе. Для продолжения резервирования измените namespace в Schedule и только затем снимите паузу.

Для платформы подготовлены Git-исходники, манифесты, сценарии, отчёт и проверенные результаты. Пользователь добавляет remote и выполняет push. Содержимое work/ и сырые резервные копии в публичный архив не входят. Для полного удаления стенда используется kind delete cluster --name practice1; это также удаляет локальный MinIO, поэтому предварительно сохраните нужные копии во внешнем S3.

Ограничения

Один узел, одна база, неизменяемый контрольный набор; без PITR, HA, нагрузочного теста и проверки полной потери кластера. MinIO Community собирается из закреплённого архивного релиза исключительно для практики. VK Cloud описан, но не проверен учётными данными пользователя. Фактический PASS относится только к выполненному локальному сценарию.

Первичные источники (проверены 02.11.2026)

1-4. Kubernetes: PV, Secret, Namespace, Job
https://kubernetes.io/docs/concepts/

5. kind v0.33.0 и образы узлов
https://github.com/kubernetes-sigs/kind/releases/tag/v0.33.0

6. Local Path Provisioner
https://github.com/rancher/local-path-provisioner

7. Velero: File System Backup
https://velero.io/docs/v1.18/file-system-backup/

8-9. Velero: hooks и restore
https://velero.io/docs/v1.18/backup-hooks/

10. AWS plugin: совместимость
https://github.com/velero-io/velero-plugin-for-aws

11. PostgreSQL: pg_dump
https://www.postgresql.org/docs/17/app-pgdump.html

12. MinIO: исходники и способ сборки
https://github.com/minio/minio

13. VK Cloud: документация S3
https://github.com/vk-cs/docs-public/tree/master/docs/ru/storage/s3

14. Docker Desktop: Windows
https://docs.docker.com/desktop/setup/install/windows-install/

Точные тематические ссылки и дополнительные пояснения находятся в docs/SOURCES.md. Реализация и журналы проекта являются источником экспериментальных результатов.